Lab 7 : A Timed Speed Boost with Sound, Particles and a HUD

Lab Overview:

In this lab you will add a **Speed Boost** power-up to the game. Power-ups float in the level. When the player touches one, it **disappears**, plays a **sound** and a **particle burst**, and makes the player run faster for a limited time. A **HUD icon** with a countdown stays on screen for as long as the boost is active. A few seconds after being collected, the power-up **reappears** so it can be collected again.

By the end of the lab you should be able to explain how a **timer** counts down with `Time.deltaTime`, how `Invoke` runs a method a few seconds later, how to build and spawn a **particle effect** prefab, how to play a sound with an **AudioSource**, why each script in a feature should have **one job**, and how to show and hide a HUD element from code.

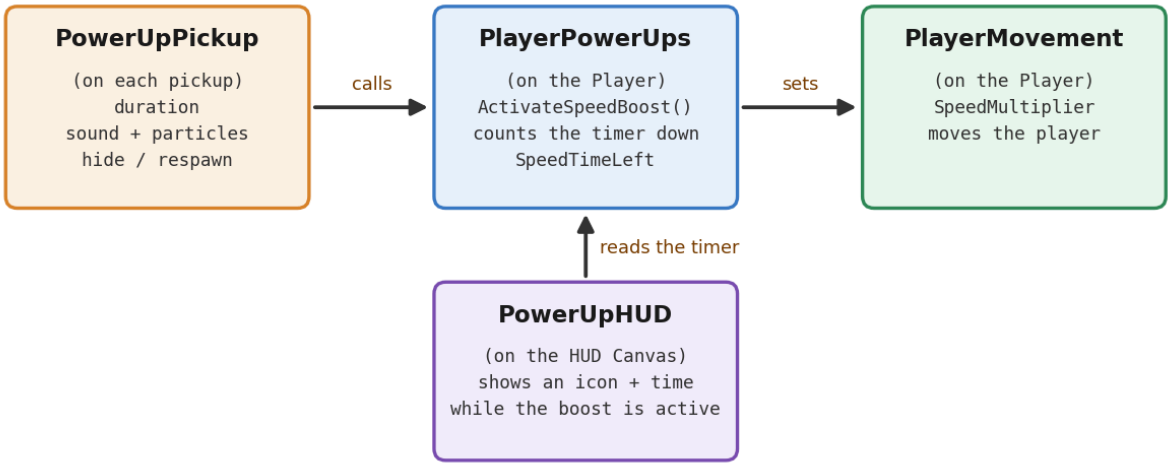


Figure 1: The four scripts in this lab and how they talk to each other

Before You Start:

1. **Get the starter project:**
    - Open the GitHub Classroom assignment (or the repo) and clone it: https://github.com/amcnabbbaltar/450_541_Lab7_2026
    - Open the project in **Unity 6**, then open **Assets/Scenes/SampleScene**.
2. **Look at what you are starting with:**
    - The Lab 7 starter contains the finished project from the previous labs. Before adding anything, find these:
    - **Assets/Scripts**: `PlayerMovement`, `PlayerAnimatorController`, `RollBehaviour`, `ScoreManager` and `ScorePickup`. Put every new script you create in this folder.
    - From Lab 6: the **Flip** double jump, the **StartMenu** and **EndScene** scenes, and the **HUD** Canvas with the score.

- **Assets/Prefab/Pickup**: the spinning coin from Lab 4. Our power-up is built the same way: an empty **parent** with the trigger and scripts, and an animated **Visuals** child.
- The **Player** in the scene is tagged **Player** and has a Rigidbody, a Capsule Collider, **Player Movement** and **Player Animator Controller**.

3. **Check that the project runs:**
   - Select the **Player** and look at its **Animator** component. **Avatar** must be **X BotAvatar**. If it shows **None** or **Missing**, click the circle next to the field and choose **X BotAvatar**.
   - Press **Play**: the character should walk, run, jump and roll, and coins should print the score in the Console.

> **Good to know:** The Player in the scene is an **instance of a prefab**. Components and values you add to it in this lab show up in **bold** with a blue line in the Inspector. These are called **overrides** and they are fine.

Step-by-Step Solution:

Task 1: Give PlayerMovement a Speed Multiplier

A power-up needs something to change. We give **PlayerMovement** one simple setting, a speed multiplier, that other scripts are allowed to change. PlayerMovement doesn't know anything about power-ups, pickups or timers: it just moves at whatever multiplier it is given.

1. **Add the multiplier property:**
   - Open **PlayerMovement** (in **Assets/Scripts**). Add this property **below** the `public bool IsGrounded => isGrounded;` line:

```
// Multiplies the normal speed. 1 = normal, 1.5 = 50% faster.
// Other scripts (like PlayerPowerUps) are allowed to change it.
public float SpeedMultiplier { get; set; } = 1f;
```

   - Unlike `IsGrounded`, this property has a public `set`, because we **want** another script to change it.
   - `= 1f` gives it a starting value: by default the player moves normally.

2. **Use the multiplier:**
   - In **MovePlayer()**, multiply the speed:

```
    Vector3 targetVelocity = moveDirection * moveSpeed
                           * SpeedMultiplier;
```

   - Save and press **Play**. Nothing should have changed: the multiplier is 1.
   - Quick test: temporarily change `= 1f` to `= 2f`, save, and press Play. The player should run twice as fast. Put it back to `1f` afterwards.

> **One job per script:** We could put the power-up timer straight into PlayerMovement, but then it would grow every time we add a new power-up.

> Instead, PlayerMovement only exposes a **setting**, and a new **PlayerPowerUps** script (Task 2) decides **when** and **for how long** that setting changes. Each script has one job, which makes both easier to read, test and reuse.

3. **Check your understanding:**
    - Why does `SpeedMultiplier` have a public `set`, while `IsGrounded` only has a getter?

Because SpeedMultiplier will be modified after an event triggers while is grounded just checks the state of the player.

Task 2: Create the PlayerPowerUps Script

PlayerPowerUps lives on the Player. When it receives a speed boost, it starts a **timer**, changes the speed multiplier in PlayerMovement, and puts it back to normal when the timer runs out.

1. **Create the script:**
    - Select the **Player**, click **Add Component**, create a new script called **PlayerPowerUps**, move it into **Assets/Scripts**, and replace its contents with:

```
using UnityEngine;

// Gives the player a speed boost for a limited time.
public class PlayerPowerUps : MonoBehaviour
{
    [SerializeField] private float boostMultiplier = 1.5f; // 50% faster

    // Seconds of boost left. The HUD reads it; only this script changes it.
    public float SpeedTimeLeft { get; private set; }

    private PlayerMovement movement;

    private void Start()
    {
        movement = GetComponent<PlayerMovement>();
    }

    // Called by the pickup. Collecting another boost restarts the timer.
    public void ActivateSpeedBoost(float duration)
    {
        SpeedTimeLeft = duration;
        movement.SpeedMultiplier = boostMultiplier;
    }

    private void Update()
    {
        if (SpeedTimeLeft <= 0f) return; // no boost running

        SpeedTimeLeft -= Time.deltaTime;

        if (SpeedTimeLeft <= 0f)
        {
            movement.SpeedMultiplier = 1f; // back to normal speed
        }
    }
}
```

2. **Understand the important lines:**
    - **SpeedTimeLeft** is the timer. Above 0 means the boost is running.
    - `{ get; private set; }` means any script can **read** it, but only PlayerPowerUps can **change** it, like `Score` in ScoreManager.
    - **Time.deltaTime** is the number of seconds since the last frame (about 0.016 at 60 FPS). Subtracting it every frame makes the timer count real seconds, whatever the frame rate.

- When the timer crosses 0, the multiplier goes back to 1 and the player runs at normal speed again.

3. **Check your understanding:**
   - What would happen if `Update()` subtracted 1 from `SpeedTimeLeft` every frame instead of `Time.deltaTime`?

It would decrement based on the players fps which would drastically change the amount of time each player has the power up.

   - `Update()` checks `SpeedTimeLeft <= 0f` twice. What is each check for?

The first if statement will end the update early if there is no powerup on the player which makes it more efficient. Second if returns the speed back to normal after it is done.

Task 3: Build the Power-Up Object

The power-up uses the same structure as the coin from Lab 4: an empty **parent** that holds the trigger and the scripts, and a **Visuals** child that we animate and hide. The parent stays active the whole time, which is what lets it play a sound and come back after it is "gone".

1. **Create the parent and the child:**
    - Right-click the Hierarchy > **Create Empty**, rename it **SpeedBoostPickup**, and place it somewhere the player can reach, with **Position Y** at 0.6.
    - Right-click **SpeedBoostPickup** > **3D Object > Cube** and rename it **Visuals**. Set **Rotation** to 45, 0, 45 and **Scale** to 0.4, 0.4, 0.4 so it looks like a diamond.
    - Remove the **Box Collider** from **Visuals** (right-click the component header > **Remove Component**). The parent will have the collider.
    - In **Assets/Material**, create a material (**Create > Material**) called **SpeedBoostMaterial**, make it bright yellow, and drag it onto **Visuals**.
2. **Make the parent a trigger:**
    - Select **SpeedBoostPickup** and add a **Sphere Collider**. Check **Is Trigger** and set **Radius** to 0.6.

> **Trigger reminder:** A trigger is not solid. The player walks through it, and Unity calls `OnTriggerEnter` on our script the moment the player enters it. Triggers only fire if one of the two objects has a Rigidbody, and our Player does.

3. **Add an Audio Source to the parent:**
    - With **SpeedBoostPickup** selected, click **Add Component > Audio > Audio Source**.
    - **Uncheck Play On Awake**. Leave **Audio Resource** empty: the script will choose the sound.
    - Leave **Spatial Blend** at 0 (2D) so the sound has the same volume wherever the camera is.
    - Check that the **Camera** object in the scene has an **Audio Listener** component: it is the "ears" of the scene.

4. **Animate the Visuals:**
    - Select **SpeedBoostPickup** and open **Window > Animation > Animation**. Click **Create** and save the clip as **PowerUpFloat** in **Assets/CoinAnimation**.
    - Click **Add Property** > **Visuals > Transform** and add **Position** and **Rotation**.
    - Turn on **Record**. At **0:30** set Visuals **Position Y** to 0.3. At **1:00** set Visuals **Rotation Y** to 360. Turn **Record** off.
    - Select **PowerUpFloat** in the Project window and make sure **Loop Time** is checked.

- Check the property names read **Visuals : Position** and **Visuals : Rotation**, not SpeedBoostPickup. We animate the **child** so every copy floats around its own spot.

Task 4: Get a Pickup Sound

The repo has no sounds yet, so you will find one yourself, the way you would for your own game.

1. **Find a sound online:**
    - Go to **kenney.nl/assets** and look at the free audio packs (for example the interface, digital or impact sound packs). Kenney's assets are **CC0**: free to use with no conditions.
    - **freesound.org** has many more sounds, but each one has its own licence. Read it before you download.
    - Pick a short, bright sound that says "power-up!" (a chime, a rising blip, a sparkle). Under one second is best. Download it as **.wav** or **.ogg**.

> **Check the licence:** Sounds found through a normal web search are usually **not** free to use. **CC0** means you can use the file with no conditions. **CC BY** means you can use it if you **credit the author**. Write your sound's details in the answer box below.

    - Which sound did you choose? Give its file name, author, website and licence.

Coin Pickup.mp3 by Phenala from freesound.org. CC0

2. **Import it:**
    - In the Project window, create a folder **Assets/Audio**.
    - Drag the sound file from your computer into **Assets/Audio** and rename it **PowerUpSound**.
    - Select it and click the play button in the Inspector preview to hear it.

> **AudioClip vs. AudioSource:** The sound file is an **AudioClip**: just the data. An **AudioSource** is the speaker that plays clips. In Task 6, our script will ask the pickup's AudioSource to play this clip once.

Task 5: Create a Particle Burst Effect

A **Particle System** spawns lots of tiny images (particles) and moves them according to the settings you choose: sparks, smoke, rain, magic. We will make a short burst of sparkles, save it as a prefab, and spawn a copy every time a power-up is collected.

1. **Create the Particle System:**
    - In the Hierarchy, right-click > **Effects > Particle System**. Rename it **PowerUpBurst**.
    - By default it shoots a constant stream of white particles upward. You will now turn that into a one-shot burst.
2. **Main module (the top section of the Particle System component):**
    - **Duration**: 0.5. **Looping**: **unchecked**. The effect plays once and stops.
    - **Start Lifetime**: 0.6. Each particle lives 0.6 seconds.
    - **Start Speed**: 4. **Start Size**: 0.15.
    - **Start Color**: yellow (match your pickup).
    - **Gravity Modifier**: 0.5, so the sparkles fall a little.
    - **Simulation Space**: **World**. **Play On Awake**: **checked**, so it plays as soon as it is spawned.
    - **Stop Action**: **Destroy**. When the effect is finished, Unity deletes the object for us.
3. **Emission and Shape:**
    - Open the **Emission** module. Set **Rate over Time** to 0, then click **+** under **Bursts** to add one burst: **Time** 0, **Count** 40.
    - Open the **Shape** module. Set **Shape** to **Sphere** and **Radius** to 0.2, so particles fly out in every direction.
4. **Make the particles fade and shrink:**
    - Tick **Color over Lifetime**, click the gradient bar and set the **top-right** key (alpha) to 0. Particles now fade out.
    - Tick **Size over Lifetime** and choose a curve that goes **down** from 1 to 0. Particles now shrink as they fade.
    - Click **Restart** in the small **Particle Effect** panel in the Scene view to preview the burst. Tweak the values until you like it.
5. **Save it as a prefab:**
    - Create a folder **Assets/Prefab/Effects** and drag **PowerUpBurst** from the Hierarchy into it.
    - **Delete** PowerUpBurst from the Hierarchy. From now on, the script will create copies of the prefab when they are needed.

> **Why Stop Action: Destroy?** Every pickup creates a **new** effect object. Without Stop Action: Destroy, those finished effects would stay in the scene forever,

> invisible but still using memory. You can watch them appear and disappear in the Hierarchy while playing.

Task 6: Write the PowerUpPickup Script

Now we connect everything. When the player touches the pickup, the script gives the player the power-up, plays the sound and the particle burst, hides the pickup, waits, and shows it again.

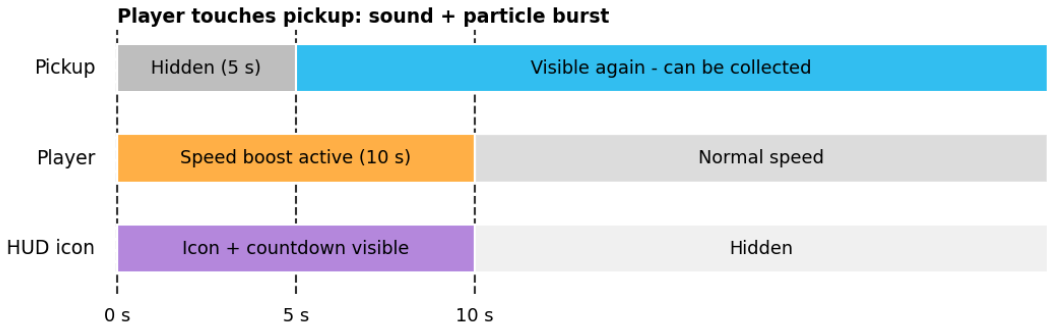


Figure 2: What happens after the player touches a pickup (default values)

1. **Create the script:**
   - Select **SpeedBoostPickup** (the parent), click **Add Component**, create a script called **PowerUpPickup**, move it into **Assets/Scripts**, and replace its contents with:

```
using UnityEngine;

// A speed boost pickup. When the player touches it, it gives the boost,
// plays a sound and particles, hides, and comes back a few seconds later.
public class PowerUpPickup : MonoBehaviour
{
    [SerializeField] private float duration = 10f;    // boost length
    [SerializeField] private float respawnDelay = 5f; // time hidden
    [SerializeField] private GameObject visuals;      // child to hide
    [SerializeField] private AudioClip pickupSound;
    [SerializeField] private GameObject pickupEffect; // particle prefab

    private void OnTriggerEnter(Collider other)
    {
        // Only the player has a PlayerPowerUps component
        PlayerPowerUps powerUps = other.GetComponent<PlayerPowerUps>();
        if (powerUps == null) return;

        powerUps.ActivateSpeedBoost(duration);

        // Sound and particles
        GetComponent<AudioSource>().PlayOneShot(pickupSound);
        Instantiate(pickupEffect, transform.position, Quaternion.identity);

        // Disappear, and call Reappear() after respawnDelay seconds
        visuals.SetActive(false);
        GetComponent<Collider>().enabled = false;
        Invoke(nameof(Reappear), respawnDelay);
    }
```

```
    private void Reappear()
    {
        visuals.SetActive(true);
        GetComponent<Collider>().enabled = true;
    }
}
```

2. **Fill in the Inspector:**
    - **Duration**: 10. **Respawn Delay**: 5.
    - **Visuals**: drag the **Visuals** child in.
    - **Pickup Sound**: drag **PowerUpSound** from **Assets/Audio**.
    - **Pickup Effect**: drag the **PowerUpBurst** prefab from **Assets/Prefab/Effects**.
3. **Understand the important lines:**
    - `other.GetComponent<PlayerPowerUps>()` looks for PlayerPowerUps on whatever touched the trigger. Only the Player has one, so coins, walls and the floor are ignored.
    - `PlayOneShot(pickupSound)` plays the clip once on the pickup's Audio Source.
    - `Instantiate(prefab, position, rotation)` creates a **new copy** of a prefab in the scene. Here it puts the particle burst exactly where the pickup is.
    - `Invoke(nameof(Reappear), respawnDelay)` asks Unity to call `Reappear()` in 5 seconds. Meanwhile the game keeps running normally. `nameof` turns the method name into text, and gives a compile error if you mistype it.

> **Why hide the child instead of the whole pickup?** `Destroy(gameObject)` (what the coin does) removes the pickup for good, so it could never come back. Turning off the whole pickup would also turn off its Audio Source, cutting the sound. Hiding only the **Visuals** child and disabling the **collider** makes the pickup look gone and stop working, while the parent stays active to play the sound and bring it back.

4. **Test the Speed Boost:**
    - Press **Play** and run through the pickup. You should hear the sound, see the burst, and the pickup should vanish.
    - The player now runs faster for 10 seconds, then goes back to normal speed.
    - After 5 seconds the pickup reappears and can be collected again. Collecting it again while the boost is active restarts the 10 seconds.

> **Nothing happens?** Check that the Sphere Collider has **Is Trigger** ticked and that the Player has the **PlayerPowerUps** component.
>
> **Burst but no sound?** Check **Pickup Sound** is filled, the Audio Source volume isn't 0, and the Game view's **Mute Audio** button is off.
>
> **Sound but no burst?** Check **Pickup Effect** is filled with the **prefab** from the Project window, not an object from the Hierarchy, and that the prefab has **Play On Awake** checked.

> **Error: MissingComponentException (AudioSource)?** The pickup has no **Audio Source**. Go back to Task 3 and add one.
>
> **The pickup never comes back?** The **Visuals** slot probably points at the parent instead of the child.

5. **Make it a prefab and place a few:**
    - Drag **SpeedBoostPickup** from the Hierarchy into **Assets/Prefab** to make it a prefab.
    - Place two or three copies around the level, for example at the start of a long straight path where the extra speed is useful.

6. **Check your understanding:**
    - Why is the boost timer counted down in PlayerPowerUps instead of in the pickup? (Hint: what would happen to the boost if the pickup respawned, or was destroyed?)

      Because the lifecycle is independent from the lifecycle of the pickup. If the pickup is destroyed with the countdown in the pickup script, the game would kill the script and the game would break

    - The pickup creates a new PowerUpBurst every time it is collected. Why don't the finished effects pile up in the Hierarchy?

      Because they automatically destroy and respawn to avoid object cluttering.

Task 7: Show the Speed Boost on the HUD

The player needs to know the boost is active and how long it will last. We will add a small **icon with a countdown** in the top-left corner of the screen. It is visible **for the entire time** the boost is active, and hidden the rest of the time.



Figure 3: HUD layout while the Speed Boost is active

1. **Find an icon online:**
    - Open **game-icons.net** (simple icons, CC BY 3.0 licence) or **kenney.nl/assets** (UI and icon packs, CC0 licence).
    - Find an icon that says **speed**: a lightning bolt, a running shoe, wind lines...
    - Download it as a **PNG** with a **transparent background**, in **white** if the site lets you choose colours. 256 x 256 is plenty. Avoid **SVG** files: Unity can't use them as UI images without an extra package.
    - Which icon did you choose? Give its name, author, website and licence.

      https://game-icons.net/1x1/lorc/batteries.html#download by Lorc under CC BY 3.0

2. **Import the icon as a sprite:**
    - Create a folder **Assets/UI**, drag the PNG into it and rename it **SpeedIcon**.
    - Select it. In the Inspector set **Texture Type** to **Sprite (2D and UI)**, keep **Sprite Mode** on **Single**, make sure **Alpha Is Transparency** is checked, and click **Apply**.
    -

> **Texture vs. sprite:** A **texture** is just the grid of pixels in the image file. A **sprite** is a 2D graphic Unity makes from a texture. UI Images can only draw sprites, so every picture you want on a Canvas has to be imported as a sprite first.

3. **Find the HUD Canvas:**
    - Use the **HUD** Canvas you made in Lab 6, with the **ScoreText** in the top-left corner. The new icon goes just **below** the score.
    - If your scene has no HUD, create one: right-click the Hierarchy > **UI > Canvas**, rename it **HUD**, and in the **Canvas Scaler** set **UI Scale Mode** to **Scale With Screen Size**, **Reference Resolution** to **1920 x 1080**, and **Match** to **0.5**.

> **Why Scale With Screen Size?** With the default **Constant Pixel Size**, an icon that looks fine at 1920 x 1080 looks tiny on a 4K screen. Scale With Screen Size designs the UI for one reference resolution and scales it for every other screen.

4. **Build the Speed Boost indicator:**
    - Right-click **HUD** > **Create Empty**, rename it **SpeedBoostIndicator**. Anchor preset: hold **Alt + Shift** and click **top-left**. **Pos X** 40, **Pos Y** -130, **Width** 160, **Height** 64. That puts it just under the score.
    - Right-click **SpeedBoostIndicator** > **UI > Image**, rename it **Icon**. Anchor preset **Alt + Shift** + **middle-left**, **Width** 56, **Height** 56. Drag your **SpeedIcon** sprite into **Source Image**, tick **Preserve Aspect**, and set **Color** to yellow.
    - Right-click **SpeedBoostIndicator** > **UI > Text - TextMeshPro**, rename it **TimeText**. If Unity asks, click **Import TMP Essentials**. Anchor preset **Alt + Shift** + **middle-left**, **Pos X** 70, **Width** 90, **Height** 64. Text **10s**, Font Size 36, **Bold**, vertical alignment **middle**.
    - If your icon isn't white, leave the Image **Color** white so it keeps its own colours.

> **Anchors in one sentence:** The **anchor** is the point of the screen an element is attached to. An indicator anchored to the top-left keeps the same distance from that corner, whatever the size of the window.

5. **Create the PowerUpHUD script:**
    - Select **HUD**, click **Add Component**, create a script called **PowerUpHUD**, move it into **Assets/Scripts**, and replace its contents with:

```
using UnityEngine;
using TMPro; // TMP_Text

// Shows the speed boost icon and seconds left while the boost is on.
public class PowerUpHUD : MonoBehaviour
{
    [SerializeField] private PlayerPowerUps powerUps;
    [SerializeField] private GameObject speedIndicator; // icon + time
    [SerializeField] private TMP_Text speedTimeText;

    private void Update()
```

```
    {
        float timeLeft = powerUps.SpeedTimeLeft;

        // Visible for the whole boost, hidden the rest of the time
        speedIndicator.SetActive(timeLeft > 0f);

        // CeilToInt rounds 6.2 up to 7, so we never show 0s too early
        speedTimeText.text = Mathf.CeilToInt(timeLeft) + "s";
    }
}
```

- Fill in the three slots: **Power Ups** (drag the **Player**, Unity picks its PlayerPowerUps component), **Speed Indicator** (SpeedBoostIndicator) and **Speed Time Text** (its TimeText child).
- The HUD only **reads** the timer. PlayerPowerUps owns it, so the icon and the boost always start and end at the same moment.
- The indicator is hidden with `SetActive`. That is safe here because PowerUpHUD is on the **HUD** object, not on the indicator, so its `Update()` keeps running while the indicator is hidden.

6. **Test the HUD:**
   - Press **Play**. The icon is not visible.
   - Collect a Speed Boost: the icon appears and counts down from **10s**. It disappears exactly when the player slows down again.
   - Collect another boost while the first is active: the countdown jumps back to **10s**.

> **NullReferenceException in PowerUpHUD?** One of the three slots is empty. The error message gives the line number: check which variable that line uses.
>
> **The icon is a white square?** Its **Source Image** is empty, or the PNG has no transparent background.
>
> **Can't drag the PNG into Source Image?** Its **Texture Type** is still **Default**. Set it to **Sprite (2D and UI)** and click **Apply**.

7. **Check your understanding:**
   - The icon must stay visible for the **entire** time the boost is active. Which value decides that, and which script owns it?

Power Up Pickup. The powerup.

- Why is it a good idea for the HUD to only **read** `SpeedTimeLeft` instead of keeping its own timer?

Prevents desynchronization.

Optional Challenges:

- **Warning blink:** make the icon blink during the last 3 seconds of the boost.
- **Clock icon:** put a dark copy of the icon on top of it, set its **Image Type** to **Filled** with **Fill Method Radial 360**, and set its `fillAmount` from code so the icon drains like a clock.
- **Respawn effect:** play a smaller particle burst and a different sound when the pickup reappears.
- **Speed trail:** add a **Trail Renderer** to the Player and only enable it while the boost is active.
- **Faster animation:** the character slides a little at 1.5x speed because the Running clip plays at normal speed. Add a Float parameter to the Animator and use it as the Idle Walk Run state's **Speed Multiplier**.

Final Steps:

1. **Test Your Project:**
   - Press **Play**. Collect a Speed Boost and check: the sound plays, the burst appears, and the pickup vanishes and comes back 5 seconds later.
   - The player runs faster for 10 seconds, then returns to normal speed.
   - The HUD icon is visible for the whole 10 seconds and hides when the boost ends.
   - Collect a boost while one is active: the timer restarts at 10 seconds.
   - Check that everything from the starter project still works: walking, running, jumping, falling, rolling and coins.
2. **Submission:**
   - Save your fork of the repo.
   - Submit your Assets folder zipped with this filled in and saved document in the assets folder.